Programação Aplicada em .Net

Aula 5 – Arquitetura, Evolução de APIs e Testes de Integração

Prof. Me. Lucas Teodoro dos Santos

UNIP

19 de Outubro de 2026

Roteiro da Aula

1. Arquitetura Aplicada
2. Contrato e Evolução de APIs
3. Qualidade Contínua

UNIP
UNIVERSIDADE PAULISTA

Controller Responsáveis por Todas as Operações

Imagine um endpoint para cadastrar um produto. Dentro do Controller, são realizadas todas as etapas:

1. Receber os dados da requisição;
2. Verificar se o preço é válido;
3. Consultar se o produto já existe;
4. Salvar o produto no banco de dados;
5. Retornar a resposta HTTP.

Quando vários endpoints seguem esse modelo, o Controller acumula responsabilidades.

Qual é o problema?

Uma alteração na regra de cadastro ou no acesso ao banco exige modificar o mesmo arquivo que recebe e responde às requisições.

UNIVERSIDADE PAULISTA

Divisão das Responsábilidades – Controller e Service

Controller

Responsável por receber as requisições HTTP, obter os parâmetros da URL, os cabeçalhos e os dados enviados no corpo da requisição. Também define os códigos de status, os cabeçalhos e o conteúdo das respostas HTTP.

Service

Responsável por coordenar os casos de uso e aplicar as regras da aplicação. Organiza as etapas necessárias para executar uma operação, verifica as condições para sua realização e garante o cumprimento das regras do negócio.

UNIP
UNIVERSIDADE PAULISTA

Divisão das Responsábilidades – DTO e Entidade

DTO

Responsável por definir a estrutura dos dados de entrada e saída de uma operação. Delimita quais informações podem ser recebidas ou apresentadas, evitando o transporte de dados desnecessários e a exposição de informações internas.

Entidade

Responsável por representar um conceito do domínio, com identidade, atributos e comportamentos próprios. Reúne as regras que garantem a validade de seu estado, como impedir um preço negativo ou uma alteração incompatível com sua situação atual.

UNIP
UNIVERSIDADE PAULISTA

Divisão das Responsábilidades – Infraestrutura

Persistência de Dados

Responsável por implementar consultas, inserções, atualizações e exclusões no banco de dados. Inclui a configuração da conexão, os mapeamentos e os mecanismos necessários para armazenar e recuperar as informações.

Modularização

Modularizar significa organizar o código em partes com responsabilidades bem definidas, facilitando sua compreensão, manutenção e evolução.

Em uma API, essa organização pode começar pela divisão em pastas:

- `Controllers`: Recebimento de requisições e definição das respostas HTTP;
- `Services`: Coordenação das operações e aplicação das regras da aplicação;
- `DTOs`: Definição dos dados de entrada e saída;
- `Models`: Representação das entidades e dos conceitos do domínio;
- `Data`: Configuração do `DbContext`, mapeamentos e persistência.

Além das pastas

Criar pastas não é suficiente: o código de cada parte deve respeitar a responsabilidade atribuída a ela.

UNIVERSIDADE PAULISTA

Responsabilidades do Controller

```
[HttpGet("{id}")]
public async Task<ActionResult<LivroResponse>> Buscar(int id)
{
    var livro = await _livroService.BuscarAsync(id);

    if (livro is null)
        return NotFound();

    return Ok(livro);
}
```

O Controller recebe o identificador e traduz o resultado da operação em uma resposta HTTP: `404 Not Found` quando o livro não é encontrado ou `200 OK` quando existe.

Seu limite é o tratamento da comunicação HTTP. Consultas ao banco, regras de negócio e decisões sobre como executar a operação devem ficar fora dele.

UNIP
UNIVERSIDADE PAULISTA

Responsabilidade do Service

```
1 public async Task<LivroResponse?> BuscarAsync(int id)
2 {
3     var livro = await _context.Livros.FindAsync(id);
4
5     if (livro is null)
6         return null;
7
8     return new LivroResponse(livro.Id, livro.Titulo, livro.Autor);
9 }
```

O Service coordena a busca do livro e prepara os dados de retorno. Neste exemplo, consulta o banco com EF Core e retorna `null` quando o registro não existe.

Também concentra as regras da aplicação quando a operação exige essas verificações. Seu limite é a execução da operação: não deve definir códigos de status HTTP nem retornar `Ok()` ou `NotFound()`.

DTOs e Entidades

Entidade

Representa um elemento do negócio, como Livro ou Leitor, com seus dados e regras próprias. Pode conter informações internas necessárias ao funcionamento da aplicação, que não devem ser expostas ao cliente.

DTO

Define os dados recebidos ou enviados pela API em uma operação. Sua estrutura é definida conforme a finalidade da comunicação, podendo incluir apenas parte dos dados de uma entidade ou reunir informações de várias entidades.

Exposição de dados

Uma entidade de usuário pode possuir o campo `SenhaHash`. O DTO de resposta deve apresentar somente os dados necessários, como identificador e nome, sem expor esse campo interno.

UNIVERSIDADE PAULISTA

Inversão de Controle

A **Inversão de Controle (IoC)** é um princípio que transfere o controle de uma atividade da aplicação para um mecanismo externo, como um framework.

No gerenciamento de dependências, isso significa que uma classe deixa de controlar a criação dos objetos necessários ao seu funcionamento.

Esses objetos são chamados de **dependências**. Por exemplo, se um Controller utiliza um Service para executar suas operações, esse Service é uma dependência do Controller.

No ASP.NET Core, um **container de dependências** pode assumir a criação e o gerenciamento desses objetos.

UNIP
UNIVERSIDADE PAULISTA

Criação da dependência

Neste exemplo, o próprio Controller cria o `LivroService` utilizando `new`:

```
1 public LivrosController()
2 {
3     _livroService = new LivroService();
4 }
```

O Controller controla qual objeto será criado e como ele será construído.

Se o construtor do Service passar a exigir uma conexão com o banco ou outra dependência, será necessário alterar esse código para fornecer os novos argumentos.

Além de tratar as requisições HTTP, o Controller assume a responsabilidade de construir sua dependência.

UNIP
UNIVERSIDADE PAULISTA

Injeção de Dependência

A **Injeção de Dependência (DI)** é uma técnica em que uma classe recebe externamente os objetos necessários ao seu funcionamento.

Uma das formas mais utilizadas é a **injeção pelo construtor**: as dependências são declaradas como parâmetros e fornecidas no momento da criação do objeto.

Dessa forma, a classe declara o que precisa e utiliza os objetos recebidos, sem assumir sua criação.

A injeção pode ser feita manualmente ou por um container. No ASP.NET Core, o container integrado automatiza esse processo.

UNIP
UNIVERSIDADE PAULISTA

Dependência pelo Construtor

Agora, o Controller recebe o `LivroService` como parâmetro do construtor:

```
1 public LivrosController(LivroService livroService)
2 {
3     _livroService = livroService;
4 }
```

O parâmetro `livroService` declara a dependência necessária. A atribuição ao campo `_livroServ` permite utilizá-la nos métodos do Controller.

A criação do Service fica sob responsabilidade externa. Na aplicação configurada com DI, o ASP.NET Core providencia essa dependência ao criar o Controller.

Registro no container

Para que o container forneça o `LivroService`, o serviço deve ser registrado no `Program.cs`, antes de `builder.Build()`:

```
1 builder.Services.AddScoped<LivroService>();
```

Esse registro informa que o container pode criar e fornecer objetos do tipo `LivroService`.

O tempo de vida `Scoped` permite reutilizar a mesma instância dentro de um escopo. Em uma API ASP.NET Core, normalmente cada requisição HTTP possui seu próprio escopo.

Caso o Service possua outras dependências no construtor, elas também precisam estar disponíveis para resolução pelo container.

Inversão de Controle
É o princípio aplicado quando o Controller deixa de controlar a criação de sua dependência e essa responsabilidade passa para um mecanismo externo.

Injeção de Dependência
É a técnica utilizada para fornecer o Service ao Controller. No exemplo, o objeto é entregue pelo construtor.

Container de dependências
É o mecanismo do ASP.NET Core que resolve, cria e gerencia as dependências conforme os registros e os tempos de vida configurados.

Limites da Aplicação

Boundaries são limites que definem o que cada parte da aplicação deve fazer e quais detalhes precisa conhecer.

- **Controller**: Trata requisições e respostas HTTP;
- **Service**: Coordena operações e aplica regras da aplicação;
- **Model**: Representa entidades e suas regras próprias;
- **Infraestrutura**: Acessa bancos e serviços externos.

Exemplo

O Service verifica se um livro pode ser emprestado. Ele informa o resultado da operação, mas não retorna `Ok()` ou `BadRequest()`: essas respostas HTTP são responsabilidade do Controller.

UNIP
UNIVERSIDADE PAULISTA

Contrato da API

O **contrato de uma API** define como o cliente deve enviar uma requisição e o que pode esperar como resposta.

- **Rota e método HTTP**: Endereço e operação disponíveis;
- **Dados de entrada**: Parâmetros, campos e formatos aceitos;
- **Códigos de status**: Indicação do resultado da requisição;
- **Dados de saída**: Estrutura, tipos e significado dos campos retornados.

Exemplo

Se o cliente espera o campo `titulo` na resposta e a API passa a retornar `nome`, ele pode deixar de exibir essa informação corretamente.

Compatibilidade do Contrato

Uma mudança é **compatível** quando os clientes atuais continuam utilizando a API sem precisar alterar seu código.

Mudanças geralmente compatíveis

- Adicionar um endpoint;
- Adicionar um parâmetro opcional, preservando o comportamento anterior quando ele não é informado;
- Adicionar um campo na resposta, desde que os clientes aceitem campos desconhecidos.

Mudanças incompatíveis

- Remover ou renomear um campo utilizado pelo cliente;
- Alterar o tipo ou o significado de um campo;
- Tornar obrigatório um dado antes opcional;
- Alterar ou remover uma rota utilizada pelo cliente.

UNIP
UNIVERSIDADE PAULISTA

Versionamento da API

O **versionamento** permite disponibilizar contratos diferentes de uma API, oferecendo tempo para que os clientes se adaptem a mudanças incompatíveis.

Uma nova versão pode ser necessária ao remover campos, alterar tipos de dados ou modificar o formato de uma resposta utilizada pelos clientes.

Correções e adições compatíveis geralmente podem ser realizadas na versão atual, desde que preservem o comportamento esperado pelos clientes.

Versão do contrato

A versão da API identifica o contrato oferecido ao cliente. Nem toda alteração no código exige uma nova versão da API.

UNIP
UNIVERSIDADE PAULISTA

Versão na URL

Uma forma de identificar a versão é incluí-la na rota. O cliente escolhe qual contrato utilizar pelo endereço:

```
GET /api/livros/1
```

```
GET /api/v2/livros/1
```

A primeira versão retorna os dados do livro sem o ano de publicação:

```
{ "id": 1, "titulo": "Dom Casmurro", "autor": "Machado de Assis" }
```

A segunda versão acrescenta o campo `anoPublicacao`:

```
{ "id": 1, "titulo": "Dom Casmurro", "autor": "Machado de Assis", "anoPublicacao": 1899 }
```

A V1 mantém seu contrato, enquanto a V2 disponibiliza também o ano de publicação.

UNIP
UNIVERSIDADE PAULISTA

Migração entre versões

Publicar a `v2` não atualiza automaticamente os clientes que utilizam a `v1`. Eles precisam adaptar seu código e passar a utilizar a nova rota.

- Manter a versão anterior durante o período de migração;
- Documentar as diferenças e orientar a atualização dos clientes;
- Comunicar a descontinuação e o prazo de encerramento da versão antiga;
- Acompanhar o uso antes de retirar a versão anterior.

Incluir `v2` na URL apenas identifica a versão. A implementação precisa oferecer o comportamento e os dados definidos para esse contrato.

UNIP
UNIVERSIDADE PAULISTA

Qualidade contínua na API

Uma API continua evoluindo após sua implementação. Novos recursos, correções e alterações nas regras de negócio podem afetar funcionalidades que já estavam funcionando.

A **qualidade contínua** envolve verificar a aplicação ao longo dessa evolução, buscando identificar problemas antes que afetem seus usuários.

Entre as práticas utilizadas, destacam-se:

- **Testes automatizados**: Verificam se os comportamentos esperados são preservados;
- **Análise estática**: Identifica possíveis problemas no código sem executá-lo;
- **Revisão de código**: Permite avaliar alterações antes de incorporá-las;
- **Integração contínua**: Automatiza a compilação e as verificações a cada alteração integrada.

UNIP
UNIVERSIDADE PAULISTA

Por que Automatizar os Testes?

Verificar uma API manualmente pelo Swagger ajuda a explorar seus endpoints e conferir suas respostas.

Porém, repetir todas essas verificações a cada mudança exige tempo e pode deixar problemas passarem despercebidos.

Os **testes automatizados** transformam cenários e resultados esperados em código que pode ser executado repetidamente.

Exemplo

Após alterar a V2, os testes podem verificar se a V1 continua respondendo com sucesso e mantendo os campos esperados pelos clientes.

Uma alteração que prejudica um comportamento anteriormente funcional é chamada de **regressão**.

Teste Unitário e Teste de Integração

Os testes podem verificar uma parte isolada da aplicação ou a colaboração entre diferentes componentes.

Teste unitário	**Teste de integração**
Verifica uma unidade de comportamento de forma isolada. Exemplo: conferir o resultado de uma regra de cálculo. Normalmente evita dependências externas, como banco de dados e serviços remotos.	Verifica o funcionamento conjunto de componentes. Exemplo: enviar uma requisição HTTP e conferir a resposta da API. Pode envolver roteamento, Controller, Service, serialização JSON e banco de teste.

Nesta demonstração, o foco será nos **testes de integração dos endpoints da API**.

UNIP
UNIVERSIDADE PAULISTA

O que um Teste de Integração Verifica?

Ao enviar uma requisição para `GET /api/livros`, o teste exercita os componentes envolvidos no atendimento desse endpoint.

Na implementação apresentada, esse caminho envolve:

1. O roteamento direciona a requisição ao Controller;
2. O Controller solicita os dados ao Service;
3. O Service obtém os livros da fonte de dados configurada;
4. A API produz uma resposta HTTP com conteúdo JSON.

O teste compara a resposta recebida com o resultado esperado, verificando o **status HTTP** e a **presença dos campos**.

Essas verificações avaliam o resultado do fluxo, sem conferir separadamente cada chamada interna.

UNIP
UNIVERSIDADE PAULISTA

Preparação do Projeto de Testes

Para organizar a demonstração, o projeto de testes será criado na **mesma solução da API**.

No Visual Studio:

1. No **Gerenciador de Soluções**, clique com o botão direito na solução;
2. Selecione **Adicionar** $\rightarrow$ **Novo Projeto**;
3. Escolha o modelo **Projeto de Teste xUnit**;
4. Defina o nome como `TesteIntegracao`;
5. Selecione a mesma versão do .NET utilizada pela API.

A solução passa a reunir dois projetos: a **API**, que implementa os endpoints, e o **projeto de testes**, que verifica seu comportamento.

UNIP
UNIVERSIDADE PAULISTA

Pacotes Necessários

No projeto `TesteIntegracao`, acesse **Gerenciar Pacotes NuGet**.

O modelo xUnit normalmente já inclui:

- `Microsoft.NET.Test.Sdk`: permite executar e descobrir os testes;
- `xunit`: fornece atributos e verificações, como `[Fact]` e `Assert`;
- `xunit.runner.visualstudio`: integra os testes ao Visual Studio.

Instale também:

- `Microsoft.AspNetCore.Mvc.Testing`: fornece a `WebApplicationFactory` para iniciar a API nos testes.

Utilize uma versão desse pacote compatível com a versão do .NET da API.

Referência ao Projeto da API

O projeto de testes precisa de uma **referência ao projeto da API** para acessar seus tipos.

No **Gerenciador de Soluções**:

1. Expanda o projeto `TesteIntegracao`;
2. Clique com o botão direito em **Dependências**;
3. Selecione **Adicionar Referência de Projeto**;
4. Marque o projeto `API_Evolucao_TesteIntegracao`;
5. Confirme em **OK**.

A referência parte do **projeto de testes para a API**. Estar na mesma solução facilita essa configuração, mas não substitui a referência.

UNIP
UNIVERSIDADE PAULISTA

Namespaces utilizados

No arquivo `TesteIntegracao.cs`, serão utilizados:

```
1 using System.Net;
2 using System.Text.Json;
3 using Microsoft.AspNetCore.Mvc.Testing;
4 using ApiLivros.Controllers;
5 using Xunit;
```

- `System.Net`: códigos de status HTTP;
- `System.Text.Json`: leitura da resposta JSON;
- `Microsoft.AspNetCore.Mvc.Testing`: fábrica da aplicação;
- Namespace da API: acesso ao `LivrosController`;
- `Xunit`: atributos e verificações dos testes.

Construtor do Teste

```
1  public class TesteIntegracao : IClassFixture<WebApplicationFactory<LivrosController>>
2  {
3      private readonly HttpClient _client;
4
5      public TesteIntegracao(
6      WebApplicationFactory<LivrosController> factory)
7      {
8          _client = factory.CreateClient();
9      }
10 }
```

A `WebApplicationFactory` inicia a API em um servidor de teste em memória. O tipo `LivrosController` identifica o projeto que contém a aplicação.

O `IClassFixture` permite compartilhar a fábrica entre os testes da classe. O `HttpClient` envia as requisições, sem iniciar a API manualmente.

Sucesso na V1

```
1 [Fact]
2 public async Task Listar_RetornaOk()
3 {
4     var resposta = await _client.GetAsync("/api/livros");
5
6     Assert.Equal(HttpStatusCode.OK, resposta.StatusCode);
7 }
```

O atributo `[Fact]` identifica um método de teste sem parâmetros.

O `GetAsync` envia uma requisição GET. O `await` aguarda sua resposta.

O `Assert.Equal` compara o status esperado com o recebido. Se forem diferentes, o teste falha.

Sucesso na V2

```
[Fact]
public async Task ListarV2_RetornaOk()
{
    var resposta = await _client.GetAsync("/api/v2/livros");

    Assert.Equal(HttpStatusCode.OK, resposta.StatusCode);
}
```

Este teste verifica o sucesso da requisição na rota específica da V2.

Os testes das duas versões são independentes. Uma versão pode continuar funcionando enquanto a outra apresenta um problema.

O status 200 `OK` confirma o sucesso HTTP, mas não garante que os campos da resposta estejam corretos.

Campos da V1 – Contrato

```
[Fact]
public async Task Listar_RetornaCamposEsperados()
{
    var resposta = await _client.GetAsync("/api/livros");
    Assert.Equal(HttpStatusCode.OK, resposta.StatusCode);

    var texto = await resposta.Content.ReadAsStringAsync();
    using var json = JsonDocument.Parse(texto);

    var livros = json.RootElement;
    Assert.Equal(JsonValueKind.Array, livros.ValueKind);
    Assert.True(livros.GetArrayLength() > 0);

    var livro = livros[0];
    Assert.True(livro.TryGetProperty("id", out _));
    Assert.True(livro.TryGetProperty("titulo", out _));
    Assert.True(livro.TryGetProperty("autor", out _));
}
```

O teste verifica se a resposta é um array e se o primeiro livro possui os campos `id`, `titulo` e `autor`, sem exigir valores fixos.

Leitura da Resposta

O corpo da resposta é lido como texto e interpretado como um documento JSON:

```
var texto = await resposta.Content.ReadAsStringAsync();
using var json = JsonDocument.Parse(texto);

var livros = json.RootElement;
```

`RootElement` representa o elemento principal do JSON. Neste endpoint, espera-se uma lista:

```
Assert.Equal(JsonValueKind.Array, livros.ValueKind);
Assert.True(livros.GetArrayLength() > 0);

var livro = livros[0];
```

A lista precisa conter pelo menos um livro para permitir a verificação dos campos. Essa é uma condição dos dados deste teste, não uma exigência da API.

Verificação dos Campos

```
1 Assert.True(livro.TryGetProperty("id", out _));
2 Assert.True(livro.TryGetProperty("titulo", out _));
3 Assert.True(livro.TryGetProperty("autor", out _));
```

O método `TryGetProperty` procura um campo pelo nome e retorna `true` quando ele existe.

O `Assert.True` exige que essa condição seja verdadeira. Se um campo esperado desaparecer ou for renomeado, o teste falha.

A expressão `out _` descarta o valor encontrado, pois a verificação considera apenas a presença do campo.

Campos da V2 – Contrato

```
[Fact]
public async Task ListarV2_RetornaCamposEsperados()
{
    var resposta = await _client.GetAsync("/api/v2/livros");
    Assert.Equal(HttpStatusCode.OK, resposta.StatusCode);

    var texto = await resposta.Content.ReadAsStringAsync();
    using var json = JsonDocument.Parse(texto);

    var livros = json.RootElement;
    Assert.Equal(JsonValueKind.Array, livros.ValueKind);
    Assert.True(livros.GetArrayLength() > 0);

    var livro = livros[0];
    Assert.True(livro.TryGetProperty("id", out _));
    Assert.True(livro.TryGetProperty("titulo", out _));
    Assert.True(livro.TryGetProperty("autor", out _));
    Assert.True(livro.TryGetProperty("anoPublicacao", out _));
}
```

A V2 deve apresentar além dos campos anteriores o campo `anoPublicacao`.

Execução dos Testes

No Visual Studio, o **Gerenciador de Testes** permite executar os testes e acompanhar seus resultados.

Para utilizá-lo:

1. Acesse **Teste → Gerenciador de Testes**;
2. Localize os testes da classe `TesteIntegracao`;
3. Clique em **Executar Todos** para executar todos os testes;
4. Para executar apenas um método, clique com o botão direito sobre ele e selecione **Executar**.

Após a execução, o gerenciador apresenta quais testes foram **aprovados** ou **falharam**. Ao selecionar um teste com falha, é possível consultar a mensagem que explica o resultado.

Com a classe completa, são esperados **quatro testes aprovados**, desde que a API atenda às verificações.

UNIP
UNIVERSIDADE PAULISTA

Abrangência dos Testes

Os testes apresentados verificam o funcionamento integrado da API, desde a requisição HTTP até a resposta produzida.

O que foi verificado

- Retorno de 200 `OK` nas duas versões;
- Resposta organizada como uma lista;
- Presença dos campos esperados no primeiro livro.

Como ampliar a cobertura

- Validar os tipos, os valores e todos os itens retornados;
- Testar cadastros, alterações e exclusões;
- Conferir validações e cenários de erro;
- Verificar autenticação e permissões;
- Conferir a persistência e a consulta no banco de dados.

Banco de Dados Efêmero

Um **banco de dados efêmero** é criado para uso temporário e pode ser descartado após a execução dos testes.

Nos testes de integração, ele permite:

- Isolar os testes dos dados reais da aplicação;
- Preparar dados conhecidos para cada cenário;
- Descartar os dados ao finalizar a execução.

Pode ser utilizado um banco em memória, como o **SQLite em memória**, ou uma instância temporária de banco em um contêiner.

UNIP
UNIVERSIDADE PAULISTA

Analisadores Roslyn

O **Roslyn** é a plataforma de compilação do .NET para C# e Visual Basic. Seus analisadores examinam o código e apontam possíveis problemas durante a edição ou compilação.

Entre os avisos apresentados pelas ferramentas, estão:

- Variáveis declaradas e não utilizadas;
- Métodos `async` sem `await`;
- Possível acesso a uma referência nula.

O arquivo `.editorconfig` permite definir padrões de código e configurar a severidade dos diagnósticos, como sugestão, aviso ou erro.

Verificações Automatizadas

As verificações de qualidade podem ser executadas automaticamente quando alterações são enviadas ao repositório.

Esse processo faz parte da **integração contínua** e pode incluir:

- Compilação da aplicação;
- Análise do código;
- Execução dos testes automatizados.

Os resultados ajudam a equipe a identificar problemas antes de incorporar uma alteração ao projeto, complementando a revisão de código.

UNIP
UNIVERSIDADE PAULISTA

Resumo Geral da Aula

- Organização da API e divisão de responsabilidades entre Controller, Service, DTOs, entidades e infraestrutura;
- Compreensão da Inversão de Controle e da Injeção de Dependência;
- Definição do contrato da API, compatibilidade e versionamento;
- Diferenciação entre testes unitários e de integração;
- Preparação e execução de testes com xUnit e `WebApplicationFactory`, verificando status HTTP e campos das respostas;
- Introdução aos bancos efêmeros, analisadores Roslyn e verificações automatizadas.

UNIP
UNIVERSIDADE PAULISTA

Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA